

Installation and Introduction to Programming

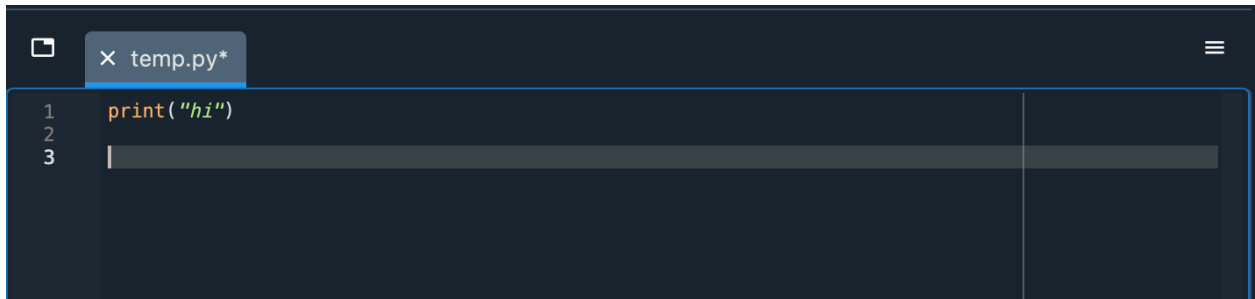
1] Download and Install Spyder IDE

Go to: <https://www.spyder-ide.org/>

And install the software appropriate for your chip/OS.

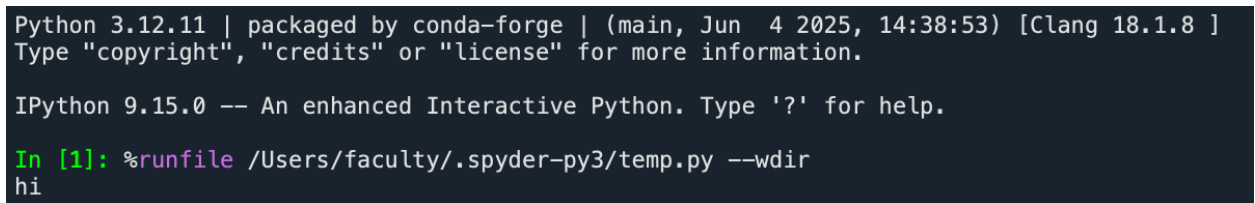
2] The application should open and you can delete all the text in green on the left.

In its place, write the following code:



```
1 print("hi")
2
3
```

Then compile and execute the code by hitting fn+f5 or by clicking the green triangle in the top menu. This will run the code and output the word “hi” to the console.

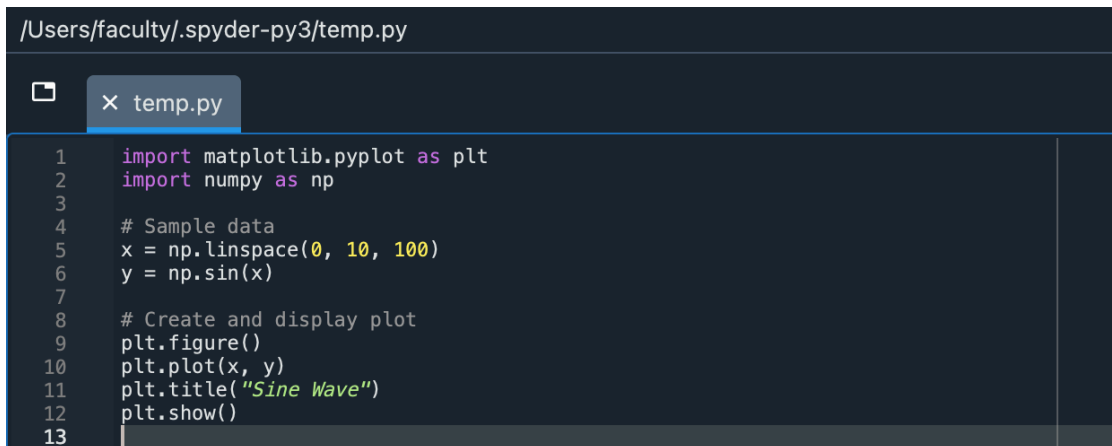


```
Python 3.12.11 | packaged by conda-forge | (main, Jun  4 2025, 14:38:53) [Clang 18.1.8 ]
Type "copyright", "credits" or "license" for more information.

IPython 9.15.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: %runfile /Users/faculty/.spyder-py3/temp.py --wdir
hi
```

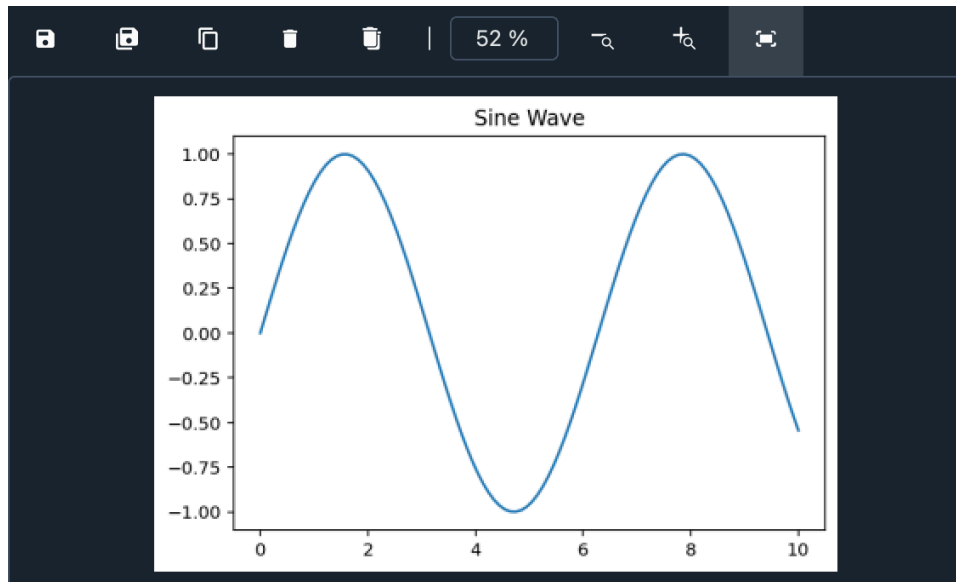
3] Plot a period or so of a sine function. Erase the text from before, and in its place, code:



```
/Users/faculty/.spyder-py3/temp.py

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Sample data
5 x = np.linspace(0, 10, 100)
6 y = np.sin(x)
7
8 # Create and display plot
9 plt.figure()
10 plt.plot(x, y)
11 plt.title("Sine Wave")
12 plt.show()
13
```

Run this code the same way as we did the other (fn + f5 or green triangle). It will output:



To the PLOT tab (be default). You can change this later if you want to be a pop-out window. For now, let's just leave it as is.

We have gone over this in class, but familiarize yourself with what each line of code is doing.

The import statements give you access to libraries; pieces of code already written that give you functions (like sine, plot, show).

The # delimiter is designed to allow “comments” to make the code more readable. Be concise with comments.

The `x` and `y =` statements assign values to `x` and `y`. Note that each of these variables is a list which contains many (in this case, 100) elements.

The `x = np.linspace(0, 10, 100)` uses the NumPy library, which contains the function `linspace` (Linear Space). The period acts as a separator between the library and the function (the function is a subset of the NumPy library (abbreviated `np`)). The `linspace` function takes three elements in its argument, the first slot is the start value of the interval, the second is the stop value of the interval, and the third is the number of points total. The function automatically determines the spacing for you. `x` is a list with values 0, 0.10101, 0.20202, ..., 10.0). If you wanted cleaner step sizes, you would have to specify 101 elements in the list construction. (You need a beginning value, end value, and a “clean” step size).

The next line assigns to the variable `y` the list that is constructed out of taking the sine of all the sequential values of `x`. Mathematically, it is equivalent to $y = \sin(x)$, but here we think of the

equals sign as an assignment operator; the thing on the right takes on a value and we assign it to the list `y` (term for term). Using the original code, `y` should have 100 elements, just like `x`.

The next few lines just display the sine wave to the screen. The **figure** function creates the canvas for plotting, the **plot** function places the information onto the canvas, the **title** does the obvious, and **show** makes the plot appear on screen.

4] Let's try to access certain elements of the list `x` and then other elements of the list `y`. We will use the syntax `x[0]` to access the first element, `x[1]` to access the second element, and so on. If we defined the list to be 100 elements long, the last element is 99; the index runs from 0 to 99.

After we run the program above, we can directly access the list `x` and its elements in the console:

```
In [41]: print(x[2])
0.20202020202020202
```

Where I've typed `print(x[2])` into the console and hit return. This doesn't run the code again, as the variables are still stored in memory. It's just accessing the value of that element and printing it to the screen.

Try to print the first and last elements of the list `x`. Do you get logical values? (You should!)

You can do this for the list `y` as well, as it is just another list in memory. Since the first element `x[0]` should be 0.0, the sine evaluation should also be zero. Check that that is true by printing the first element of `y` to the screen.

You can also add these lines of code to your program so that when your code executes, it will output the values to the screen. In the code snippet below, I've added a few print statements that will output the values of various `x` and `y` values at different points in the execution of the code. There is no point in asking the code to output a value of a variable that isn't defined yet, and depending on your compiler you might get an error. Try to print something for `x` before # sample data and check out the error.

```
import matplotlib.pyplot as plt
import numpy as np

# Sample data
x = np.linspace(0, 10, 100)
print(x[1])
y = np.sin(x)
print(y[4])

# Create and display plot
plt.figure()
plt.plot(x, y)
plt.title("Sine Wave")
plt.show()
print(x[5])
```

Exercises:

5] Using the code from 3], swap it out for cosine and plot. You should get the obvious thing.

6] Now try tangent. Change the number of points to 1000, and then plot it for just 10 points (comment on how the finite step size affects plotting of asymptotes)

7] Now force the period to be exactly 2. (Remember how? The period of a sine function $\sin(bx)$ is $2\pi/b$, so if you want the period to be 2, b must equal to π).

NumPy has a high precision definition of Pi, using “np.pi”.

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3
4  q = np.pi
5
6  # Sample data
7  x = np.linspace(0, 10, 100)
8  y = np.sin(q*x)
9
10
11 # Create and display plot
12 plt.figure()
13 plt.plot(x, y)
14 plt.title("Sine Wave")
15 plt.show()
16
```

I've defined pi in line 4, and then used it as the argument of the sine function in line 8. So now the period of the plotted sine function should be exactly 2. Run it and find out!

Now change the period to 0.2. You can do this by:

```
4  q = 10*np.pi
```

Such that the ratio is $2\pi/10\pi$ is 0.2.

Plot it and see what happens. What is going on? Instead of 10, make the multiplier 100. This sets the period to 0.02s. How about a multiplier of 50 ($T = 0.04s$)? This is an important concept called “aliasing” which you need to watch out for when generating function values.